Lecture 9

Shiven Bajpai - AI24BTECH11030
Shivram S - AI24BTECH11031

26.08.2024

Defining Performance

There is no single definition of performance. There are many different metrics and every system comes with trade-offs among these metrics. Depending on the context, different metrics may be more important and depending on the priority/weightage of these metrics, we declare one system to be more performant than the other.

In terms of computer systems or processor design, there are many metrics like speed (response time or throughput), energy, power, cost, reliability, etc. that are applicable. These metrics may sometimes conflict with one another and trade-off analysis to determine suitable operating points is a key aspect of CPU design.

For discussion in this chapter, we consider performance to be defined in terms of speed. We discuss speed related metrics now.

Response time and Throughput

- **Response time:** How long does it take to complete a task, or the total time elapsed from the start till the end of the task.
- **Throughput:** Total work done per unit time or number of tasks completed per unit time (e.g., tasks/transactions per hour)

It is important to note that while response time can affect the throughput, they are not always the same. They are correlated to some extent but for the same response time, two processors can have different throughput. One may feel faster even if it has the same response time, if it can do more tasks in parallel (e.g., via more cores).

Relative performance

We define Performance of a processor for a given program as $\frac{1}{\text{Execution Time}}$. Then if we say X is n times faster than Y, we mean

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = n$$

$$\frac{\text{Execution Time}_Y}{\text{Execution Time}_X} = n$$

Example:

If a program takes 10 seconds to run on processor A and 15 seconds to run on processor B

$$\frac{\text{Execution Time}_B}{\text{Execution Time}_A} = \frac{15}{10} = 1.5$$

So A is 1.5 times faster than B

CPU Clocking

Operation of digital hardware is governed by a constant-rate clock. The speed of this clock is measured in Hz (or MHz or GHz for faster processors). The processor executes instructions on every tick of the clock. If the clock ticks faster, the processor executes instructions faster. There are two important values associated with CPU Clocks

Clock frequency (rate): cycles per second
e.g 4.0 GHz = 4000 MHz = 4×10^9 Hz

Clock period: duration of a clock cycle
e.g $\frac{1}{4 \times 10^9 \text{Hz}} = 250 \times 10^{-12}\text{s} = 250\text{ps}$

Clock period is the reciprocal of clock frequency.

CPU Time

CPU Time refers to the total time taken by the CPU to execute a program. It can be written as:

$$\text{CPU Time} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time} = \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$

Therefore, Performance can be improved by:

- Reducing the number of clock cycles required, or
- Increasing clock rate (or reducing clock cycle time)

These are conflicting goals and it is usually not possible to achieve both goals in hardware design, and a lot of analysis and optimization of trade-offs is done by hardware designers in this aspect to determine an appropriate clock frequency.

Example:

Say you are given Computer A: 2 GHz clock, 10s CPU Time

Design a computer B aiming for 6s CPU Time, you can use a faster clock but it will cause 1.2x clock cycles due to hardware characteristics.
How fast must the Computer B clock be?

Solution:

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10\text{s} \times 2\text{ GHz} \\ &= 20 \times 10^9\text{ Hz}\end{aligned}$$

$$\begin{aligned} 6\text{s} = \text{CPU Time}_B &= \frac{\text{Clock Cycles}_B}{\text{Clock Rate}_B} \\ &= \frac{1.2 \times \text{Clock Cycles}_A}{\text{Clock Rate}_B} \end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6\text{s}} = \frac{24 \times 10^9}{6\text{s}} = 4 \text{ GHz}$$

This example shows that by increasing the clock rate appropriately, the CPU time can be reduced from 10 seconds to 6 seconds.

Instruction count and CPI

CPI is short for Cycles per Instruction and is a standard term used in CPU design and analysis.

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per instruction}$$

$$\begin{aligned} \text{CPU Time} &= \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time} \\ &= \frac{(\text{Instruction Count} \times \text{CPI})}{\text{Clock Rate}} \end{aligned}$$

Instruction Count for a program depends on the program as well as the ISA and the compiler used. Average cycles per instruction is determined by CPU Hardware.

Example:

Say you are given
Computer A: Cycle time = 250ps, CPI = 2.0
Computer B: Cycle time = 500ps, CPI = 1.2
Both use the Same ISA
Which is faster and by how much?

Solution:

$$\begin{aligned} \text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= I \times 2.0 \times 250\text{ps} = I \times 500\text{ps} \\ \text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle time}_B \\ &= I \times 1.2 \times 500\text{ps} = I \times 600\text{ps} \\ \frac{\text{CPU Time}_B}{\text{CPU Time}_A} &= \frac{I \times 600\text{ps}}{I \times 500\text{ps}} = 1.2 \end{aligned}$$

So CPU A is 1.2 times faster than B.

CPI in More Detail

Different instruction classes take different number of cycles. In such case, the total clock cycles is a weighted sum of individual CPI and instruction count.

$$\text{Clock Cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{Instruction Count}_i)$$

Weighted average CPI:

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \frac{\sum(\text{CPI} \times \text{Instruction Count})}{\sum \text{Instruction Count}}$$

Example:

You are given two alternative compiled code sequences using instructions in classes A, B and C.

Find the number of Clock cycles required for both sequences. Which sequence is faster?

Class	A	B	C
CPI for Class	1	2	3
IC in seq 1	2	1	2
IC in seq 2	4	1	1

Solution:

Sequence 1:

IC (Instruction Count) = 5

$$\begin{aligned} \text{Clock Cycles} &= 2 \times 1 + 1 \times 2 + 2 \times 3 \\ &= 10 \end{aligned}$$

$$\text{Avg CPI} = \frac{10}{5} = 2.0$$

Sequence 2:

IC = 6

$$\begin{aligned} \text{Clock Cycles} &= 4 \times 1 + 1 \times 2 + 1 \times 3 \\ &= 9 \end{aligned}$$

$$\text{Avg CPI} = \frac{9}{6} = 1.5$$

Since Sequence 2 requires less cycles to complete, we can say that Sequence 2 is faster. One can notice that despite having more number of instructions, Sequence 2 is faster. This is because it uses more instructions with lesser CPI than what is used in Sequence 1.

Performance Summary

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

or,

$$\text{CPU Time} = \text{Instruction count (IC)} \times \text{CPI} \times \text{Clock Time } (T_C)$$

The following affect performance through different aspects, as shown.

- **Algorithm**: Affects IC, possibly CPI
- **Programming Language**: Affects IC, CPI
- **Compiler**: Affects IC, CPI
- **Instruction Set Architecture**: Affects IC, CPI, T_C

T_C is the clock period of the processor.

Numericals

Question 1:

A given application written in Java runs 15 seconds on a desktop processor. A new Java compiler is released that requires only 0.6 as many instructions as the old compilers. Unfortunately it increases the CPI by 10%. How fast can we expect the application to run using this new compiler.

Solution:

$$\begin{aligned} &\text{CPU Time} = \text{IC} \times \text{CPI} \times T_C \\ \Rightarrow &15 \text{ s} = \text{I} \times \text{CPI} \times T_C \\ &\text{CPU Time with new compiler} = 0.6 \text{ I} \times 1.1 \text{ CPI} \times T_C \\ \Rightarrow &\text{CPU Time with new compiler} = 15 \text{ s} \times 0.6 \times 1.1 = 9.9 \text{ s} \end{aligned}$$

Question 2:

Assume that for a program, compiler A results in a dynamic instruction count of 10^9 and has an execution time of 1.1s while compiler B results in a dynamic instruction count of 1.2×10^9 and an execution time of 1.5s.

- Find the average CPI for each program given that the processor has a clock cycle time of 1 ns.
- A new compiler is developed that uses only 6×10^8 instructions and has an average CPI of 1.1 . What is the speedup of using this new compiler versus compiler A?

Solution:

$$\begin{aligned}\text{Clock Speed} &= \frac{1}{\text{Clock Cycle time}} = \frac{1}{10^{-9}\text{s}} \\ &= 10^9 \text{ Hz}\end{aligned}$$

$$\begin{aligned}\text{CPI}_\text{A} &= \frac{\text{Number of Cycles}_\text{A}}{\text{Number of Instructions}_\text{A}} \\ &= \frac{\text{Execution time}_\text{A} \times \text{Clock Speed}}{\text{Number of Instructions}_\text{A}} \\ &= \frac{1.1 \times 10^9}{10^9} = 1.1\end{aligned}$$

$$\begin{aligned}\text{CPI}_\text{B} &= \frac{\text{Number of Cycles}_\text{B}}{\text{Number of Instructions}_\text{B}} \\ &= \frac{\text{Execution time}_\text{B} \times \text{Clock Speed}}{\text{Number of Instructions}_\text{B}} \\ &= \frac{1.5 \times 10^9}{1.2 \times 10^9} \approx 1.25\end{aligned}$$

Let the new compiler be called C

$$\begin{aligned}\text{Execution Time} &= \frac{\text{CPI} \times \text{Number of Instructions}}{\text{Clock Speed}} \\ &= \frac{1.1 \times 6 \times 10^8}{10^9} \\ &= 0.66\text{s}\end{aligned}$$

Therefore the speedup is $\frac{1.1}{0.66} = 1.667$

Other factors affecting the performance/execution time

- **Processor and memory system**: Determine how fast instructions are executed
- **I/O System**: Determines how fast I/O operations are executed
- **Power and energy constraints**: Might restrict operating speed
- CPU Time vs System Time vs Elapsed Time

Try the `time` command in Linux with different applications.

Processor and Motherboard

Modern processors come with many pins on the back which serve as connections between the processor and the motherboard. CPUs are placed into specially designed sockets.

Manufacturing ICs

Manufacturing an IC involves many steps. In each step, defects may creep in, which decreases the yield. A rough outline of the steps involved are as follows

- First a silicon ingot (cylindrical shape) is sliced into several blank wafers by a slicer
- After passing the blank wafers through series of 20-40 processing steps, Patterned wafers are produced. Now the wafers have various transistors and other components grown over it, which is required as per the chip's functionality. Each wafer has many such chips and each of them is called a die.
- These wafers are then tested to identify defects from the manufacturing process
- Then the wafer is cut up by a Dicer into several dies and they are again tested
- Functioning dies without defects are then bonded to the package and the final packaged dies are produced
- These packaged dies are once again tested to check for any defects introuced during packaging. Only the good ones that pass the test are shipped to customers

Yield: Proportion of non-defective dies to the total number of dies manufactured

Cost of ICs

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

$$\text{Dies per wafer} \approx \frac{\text{Wafer area}}{\text{Die area}}$$

$$\text{Yield} = \frac{1}{1 + (\text{Defects per area} \times \frac{\text{Die area}}{2}))^2}$$

- Nonlinear relation to area and defect rate.
- Wafer cost and area are fixed
- Defect rate is determined by the manufacturing process
- Die area is determined by architecture and circuit design

Example:

- Wafer Diameter: 15 cm
- Wafer Cost: 12 units
- Die Area: $2.1 cm^2$
- 0.020 defects/cm^2

Find cost per die.

$$\text{Yield} = \frac{1}{\left(1 + \left(0.02 \times \frac{2.1}{2}\right)\right)^2} = 0.9593$$

$$\text{Dies per wafer} = \frac{\pi \left(\frac{15}{2}\right)^2}{2.1} \approx 84$$

$$\text{Cost per die} = \frac{12}{84 \times 0.9593} = 0.1489$$

Note: If the number of dies per wafer is fractional, we should always round it down since a fractional number does not make sense.

Seven Great Ideas

- Use *abstraction* to simplify design
- Make the *common case fast* : We don't optimize every case, we focus on the common case
- Performance via *parallelism* : We can improve performance by performing independent tasks simulaneously
- Performance via *pipelining* : The next instruction can begin executing before the previous one has finished.
- Performance via *prediction:* If we know what kind of instruction is going to come or what memory address will be accessed, something can be done to improve the performance
- *Hierarchy* of memories: e.g., cache memory in between registers and main memory
- *Dependability* via redundancy: e.g., including 8 cores instead of the required 4.